P1 : 2,5
P2 : 2,25
P3 : 2,5
P4 : 2,5
9,75

Name: Nguyen Phuong Quoc Vuong
ID: N25DFCF086

2,5 Prob 1 Given: head → 10 → 25 → 40 → 15 → Null(
Task 1: Write pseudocode to reverse it include prev, curr, next
Task 2: Draw diagram/ step-by-step pointers showing pos & value of that 3 pointers.

2,25 Prob 2: Given head → 10 → 20 → 30 → (head)
Task 1: pseudocode for InsertField(head, newValue) to insert a new node containing newValue at the proper while circuslar-still.
Task 2: Trace steps when insert newValue = 5.

2,5 Prob 3: Given double-ll contain int.
Task 1: Write pseudocode for InsertBefore(head, target, newValue) to insert new node contain newValue immediately before the first occurence of target
Task 2: List exact order of pointer assignment required & briefly justify why this avoid memory leaks or broken links

Thành Đạt

Scanned with CamScanner

- Task 3: Handle:
  1. Target at head
  2. " ~~dx~~ is 404

2/5 Prob 4:

Given: head → 2 → 5 → 8 → null

Task 1: Write SortedInsert(head, val) to put val into

Task 2: Step-by-step table: pointer interactions & condition evaluation when put val = 6 into

Problem 1:

1.25

```
reverseLL (head):
    curr = head          // var for loop
    next = null          // this to locate next node to reverse
    prev = null          // this will save previous address node
    while (curr != null):
        next = curr.next; curr.next = prev
        prev = curr;
        curr = next
    head = prev          // free the thing (var) is needed
    return head
```

Thành Đạt

Scanned with CamScanner

1.25 Task 2: Step by-step showcase:

Before reverse:

head → 2 → 5 → 8 → null

Start reverse (a ↑ b mean a is pointing to b) (LL mean linked list)

LL: head → 2 → 5 → 8 → null

↑ curr next: null prev: null

Loop started:

next = curr.next:

head → 2 → 5 → 8 → null (LL)

↑ curr ↑ next prev: null

curr.next = prev: null ← 2 5 → 8 → null (LL)

(prev) ↑ curr ↑ next

prev = curr: null ← 2 5 → 8 → null (LL)

prev = curr ↑ next

curr = next: null ← 2 5 → 8 → null (LL)

prev ↑ curr = next

Continuing looping until curr reach the tail (mean curr == null)

Receive: null ← 2 ← 5 ← 8 ← null

↑ prev

head = curr: null ← 2 ← 5 ← 8 ← head

Done!

Thành Đạt

Scanned with CamScanner

Problem 2: Task 1:

1.25

```
InsertHead (head, newValue)
    newNode = createNode (newValue) // assume this created a variable Node
    if head == Null:          // Blank list
        newNode.next = newNode
        return newNode

    tail = head
    while (tail.next != head):
        tail = tail.next      // Create variable tail.

    newNode.next = head
    tail.next = newNode

    head = newNode
    return head.
```

1.0

Task 2:

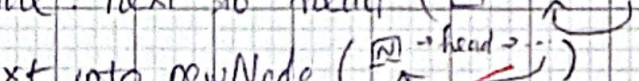
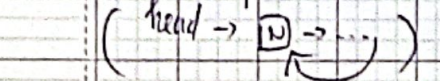
Explanation: First we use a function creating a node containing newValue as value

- In case the list is blank, create a beautiful circular: (newNode)

Thành Đạt

Scanned with CamScanner

N25DECE086

- Create a new variable "tail" to search for the last node.
- Looping until we see tail.next = head, meaning tail is the last node of the circular
- Finally, point newNode.next to head (), after that, point the tail.next into newNode (), last ~~point head into~~ let head point to newNode () and we are done.

Final result:

2.5 Problem 3:

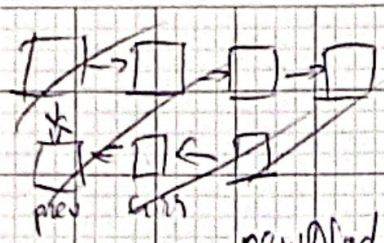
~~7.5~~ Task 1 & task 3:

InsertBefore (head, target, newValue):

```
// Edge case: Blank list
if head == null
    return head   // Target is 404 in LL
curr = head
// Finding the node containg the target
while( curr != null && curr.data != target)
    curr = curr.next
if (curr == null:) // Target is 404
    return head
```

Thành Đạt

Scanned with CamScanner




```
newNode = createNode(newValue)
// In case target is at head
if curr == head:
    newNode.next = head
    newNode.prev = null
    head.prev = newNode
    head = newNode
    return head;
// Normal case, insert before curr
newNode.prev = curr.prev
newNode.next = curr
curr.prev.next = newNode
curr.prev = newNode
return head
```

0,75 Task 2:

Exact assignment sequence

1. newNode.prev = curr.prev
2. newNode.next = curr
3. curr.prev.next = newNode
4. curr.prev = newNode

Thành Đạt

Scanned with CamScanner

Explain:

We must connect newNode's outgoing pointers first without modifying existing pointers. If curr.prev = newNod. were first, the reference to previous the preceding node would be lost, make it broken.

2.5

Problem 4:

Task 1:

```
SortedInsert(head, val):
    newNode = createNode(val)
    if (head == null or val < head.data):
        newNode.next = head
        head = newNode
        return head
    curr = head
    while (curr.next != null && curr.next.data < val):
        curr = curr.next
    newNode.next = curr.next
    curr.next = newNode
    return head
```

Thành Đạt

Scanned with CamScanner

4.25

Task 2:

Initial check: head != null

val >= head.data

set curr = head

Action	Step	curr	curr.next	curr.next != null	curr.next.data < 6
curr = curr.next	1	[2]	[5]	T	T
Stop loop at curr = [5]	2	[5]	[8]	T	F

Linking step:

1. newNode.next = curr.next

→ newNode points to [8]

2. curr.next = newNode

→ [5] point to newNode

Final list:

2 → 5 → 6 → 8 → null

Thành Đạt

Scanned with CamScanner